In [3]:

```
# ========================================
# BOOKS DATASET - DATA VISUALIZATION
# ========================================

# Step 1: Import Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline


# Step 2: Load Dataset

df = pd.read_csv('Books.csv.zip')

print("Dataset Shape:", df.shape)
display(df.head())
print(df.info())


# Step 3: Data Preprocessing

# Convert publication year into numeric format

df['Year-Of-Publication'] = pd.to_numeric(
    df['Year-Of-Publication'], errors='coerce'
)

# Remove invalid publication years

df.loc[
    ~df['Year-Of-Publication'].between(1000, 2026),
    'Year-Of-Publication'
] = np.nan

# Create a new feature for title length

df['Title_Length'] = df['Book-Title'].astype(str).str.len()


# ========================================
# MAIN DATA VISUALIZATIONS
# ========================================


# Step 4: Distribution of Book Publication Years

plt.figure(figsize=(8, 5))

plt.hist(
```

```
    df['Year-Of-Publication'].dropna(),
    bins=20,
    edgecolor='black'
)

plt.title('Distribution of Book Publication Years')
plt.xlabel('Publication Year')
plt.ylabel('Number of Books')

plt.show()


# Step 5: Publication Year Distribution by Publisher

top_publishers = df['Publisher'].value_counts().head(5).index

plt.figure(figsize=(10, 5))

sns.boxplot(
    data=df[df['Publisher'].isin(top_publishers)],
    x='Publisher',
    y='Year-Of-Publication'
)

plt.title('Publication Year Distribution by Top 5 Publishers')
plt.xlabel('Publisher')
plt.ylabel('Publication Year')

plt.xticks(rotation=45)
plt.show()


# Step 6: Top 10 Authors by Number of Books

plt.figure(figsize=(10, 5))

df['Book-Author'].value_counts().head(10).sort_values().plot(
    kind='barh'
)

plt.title('Top 10 Authors by Number of Books')
plt.xlabel('Number of Books')
plt.ylabel('Author')

plt.show()


# Step 7: Top 10 Publishers Distribution

df['Publisher'].value_counts().head(10).plot(
    kind='pie',
    autopct='%1.1f%%'
)
```

```
plt.title('Top 10 Publishers Distribution')
plt.ylabel('')

plt.show()


# Step 8: Book Title Length vs Publication Year

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df.sample(n=min(3000, len(df)), random_state=42),
    x='Year-Of-Publication',
    y='Title_Length',
    alpha=0.5
)

plt.title('Book Title Length vs Publication Year')
plt.xlabel('Publication Year')
plt.ylabel('Title Length (Characters)')

plt.show()


# Step 9: Number of Books Published by Year

books_by_year = (
    df.groupby('Year-Of-Publication')
    .size()
    .sort_index()
)

plt.figure(figsize=(10, 5))

plt.plot(
    books_by_year.index,
    books_by_year.values,
    marker='o'
)

plt.title('Number of Books Published by Year')
plt.xlabel('Publication Year')
plt.ylabel('Number of Books')

plt.grid(True)

plt.show()


# ========================================
# ADDITIONAL DATA VISUALIZATIONS
# ========================================
```

```python
# Step 10: Distribution of Book Title Lengths

plt.figure(figsize=(8, 5))

plt.hist(
    df['Title_Length'].dropna(),
    bins=20,
    edgecolor='black'
)

plt.title('Distribution of Book Title Lengths')
plt.xlabel('Title Length (Characters)')
plt.ylabel('Number of Books')

plt.show()


# Step 11: Number of Books Published by Decade

df['Decade'] = (
    (df['Year-Of-Publication'] // 10) * 10
)

books_by_decade = df['Decade'].value_counts().sort_index()

plt.figure(figsize=(10, 5))

books_by_decade.plot(kind='bar')

plt.title('Number of Books Published by Decade')
plt.xlabel('Decade')
plt.ylabel('Number of Books')

plt.xticks(rotation=45)

plt.show()


# Step 12: Missing Values in Each Column

missing_values = df.isnull().sum()

plt.figure(figsize=(10, 5))

missing_values.sort_values(ascending=False).plot(
    kind='bar',
    edgecolor='black'
)

plt.title('Missing Values in Books Dataset')
plt.xlabel('Columns')
plt.ylabel('Number of Missing Values')

plt.xticks(rotation=45)
```

```
plt.show()


# ========================================
# END OF DATA VISUALIZATION
# ========================================

print("All Data Visualizations Completed Successfully!")
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[3], line 17
     13
     14
     15 # Step 2: Load Dataset
     16
---> 17 df = pd.read_csv('Books.csv.zip')
     18
     19 print("Dataset Shape:", df.shape)
     20 display(df.head())

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\parsers\rea
ders.py:1866, in read_csv(filepath_or_buffer, sep, delimiter, header, name
s, index_col, usecols, dtype, engine, converters, true_values, false_value
s, skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_n
a, na_filter, skip_blank_lines, parse_dates, date_format, dayfirst, cache_d
ates, iterator, chunksize, compression, thousands, decimal, lineterminator,
quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_er
rors, dialect, on_bad_lines, low_memory, memory_map, float_precision, stora
ge_options, dtype_backend)
   1853 kwds_defaults = _refine_defaults_read(
   1854     dialect,
   1855     delimiter,
   (...)   1862     lineterminator=lineterminator,
   1863 )
   1864 kwds.update(kwds_defaults)
-> 1866 return _read(filepath_or_buffer, kwds)

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\parsers\rea
ders.py:423, in _read(filepath_or_buffer, kwds)
    420             return result
    422 # Create the parser.
--> 423 parser = TextFileReader(filepath_or_buffer, **kwds)
    425 if chunksize or iterator:
    426     return parser

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\parsers\rea
ders.py:2721, in TextFileReader.__init__(self, f, engine, **kwds)
   2718     self.options["has_index_names"] = kwds["has_index_names"]
   2720 self.handles: IOHandles | None = None
-> 2721 self._engine = self._make_engine(f, self.engine)

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\parsers\rea
ders.py:2988, in TextFileReader._make_engine(self, f, engine)
   2983 is_text = engine in ("python", "python-fwf") or (
   2984     engine == "c"
   2985     and self.options.get("encoding", "utf-8") not in (None, "ut
f-8")
   2986 )
   2987 mode = "r" if is_text else "rb"
-> 2988 self.handles = get_handle(
   2989     f,
   2990     mode,
```

```
   2991     encoding=self.options.get("encoding", None),
   2992     compression=self.options.get("compression", None),
   2993     memory map=self.options.get("memory_map", False),
   2994     is text=is text,
   2995     errors=self.options.get("encoding errors", "strict"),
   2996     storage_options=self.options.get("storage_options", None),
   2997 )
   2998 assert self.handles is not None
   2999 f = self.handles.handle

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\common.py:8
44, in get_handle(path_or_buf, mode, encoding, compression, memory_map, i
s_text, errors, storage_options)
    839 # ZIP Compression
    840 elif compression == "zip":
    841     # error: Argument 1 to "_BytesZipFile" has incompatible type
    842     # "Union[str, BaseBuffer]"; expected "Union[Union[str, PathLik
e[str]],
    843     # ReadBuffer[bytes], WriteBuffer[bytes]]"
--> 844     handle = _BytesZipFile(
    845         handle,  # type: ignore[arg-type]
    846         ioargs.mode,
    847         **compression_args,
    848     )
    849     if handle.buffer.mode == "r":
    850         handles.append(handle)

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\common.py:1
121, in _BytesZipFile.__init__(self, file, mode, archive_name, **kwargs)
   1115 kwargs.setdefault("compression", zipfile.ZIP_DEFLATED)
   1116 # error: No overload variant of "ZipFile" matches argument types
   1117 # "str | PathLike[str] | ReadBuffer[bytes] | WriteBuffer[bytes]",
   1118 # "str", "dict[str, Any]"
   1119 # error: Incompatible types in assignment (expression has type "Zip
File",
   1120 # base class "_BufferedWriter" defined the type as "BytesIO")
-> 1121 self.buffer: zipfile.ZipFile = zipfile.ZipFile(  # type: ignore[cal
l-overload, assignment]
   1122     file, mode, **kwargs
   1123 )

File C:\Python314\Lib\zipfile\__init__.py:1487, in ZipFile.__init__(self, f
ile, mode, compression, allowZip64, compresslevel, strict_timestamps, metad
ata_encoding)
   1485 while True:
   1486     try:
-> 1487         self.fp = io.open(file, filemode)
   1488     except OSError:
   1489         if filemode in modeDict:

FileNotFoundError: [Errno 2] No such file or directory: 'Books.csv.zip'
```

In [ ]: